

Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ТЕОРЕТИЧЕСКАЯ И СТЕНДОВАЯ СПРАВКА ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5

«Оптимизация запросов и производительность»

Назначение документа: подготовка к защите ЛР5, объяснение теории, выполненных действий, расчетов ускорения и демонстрации работы стенда.

Санкт-Петербург
2026

Содержание справки

1. Краткая суть лабораторной работы
2. Что именно было сделано в ЛР5 и зачем
3. Теоретическая база: планы, индексы, селективность
4. Запросы Q1-Q3: назначение, узкие места, оптимизация
5. Расчеты ускорения и проверка реалистичности
6. Методические риски и честная защита решений
7. На основе чего сделаны выводы
8. Команды для проверки и демонстрации в консоли
9. Короткие ответы на типовые вопросы преподавателя

1. Краткая суть лабораторной работы

ЛР5 посвящена оптимизации SQL-запросов и анализу производительности в PostgreSQL. В работе используется база данных из ЛР1 по варианту 8: документы организации и журнал операций с документами.

Основная идея эксперимента: сначала выполнить запросы без дополнительных оптимизирующих индексов и получить исходные планы выполнения, затем определить узкие места, создать индексы под реальные условия фильтрации и соединения, повторно снять планы и сравнить время выполнения.

Важная честная оговорка: использованные Q1-Q3 являются запросами, разработанными в предметной области ЛР1 для ЛР5. Формально методичка просит использовать запросы из ЛР3. Так как ЛР3 на момент выполнения была заблокирована/не завершена по внешней причине, запросы были составлены отдельно, но они покрывают требуемые классы операций: фильтрацию, JOIN, EXISTS, GROUP BY и ORDER BY.

Параметр	Значение
СУБД	PostgreSQL 16.13
Среда	WSL Ubuntu + Docker + psql
База данных	subd_lab5
Контейнер	subd_lab5_postgres
Основные таблицы	documents, document_operations
Размер данных	100000 документов, 300000 операций
Запросы	Q1, Q2, Q3
Метод измерений	5 запусков до/после оптимизации + прогревочный запуск

2. Что именно было сделано в ЛР5 и зачем

Шаг	Что сделали	Зачем
1	Создали отдельную папку Lab5 и отдельный docker-compose.	Чтобы ЛР5 была изолирована от ЛР1, ЛР3 и ЛР4, а результаты можно было воспроизвести.
2	Подняли PostgreSQL 16 на порту 25432, БД subd_lab5.	Чтобы получить независимый стенд для экспериментов с планами и индексами.
3	Создали таблицы documents и document_operations по схеме ЛР1.	Методичка требует выполнять ЛР5 над базой, разработанной в ЛР1.
4	Заполнили documents 100000 строками, document_operations 300000 строками.	Чтобы получить объем данных больше минимальных 10000 строк и увидеть изменение планов.
5	Сгенерировали распределение по типам документов, сотрудникам, подразделениям, датам и секретности.	Чтобы данные не были полностью искусственно однообразными и имели селективность.
6	Сняли исходные планы EXPLAIN ANALYZE для Q1-Q3 без оптимизирующих индексов.	Чтобы определить исходные узкие места: Seq Scan, Parallel Seq Scan, Hash Join/Semi Join.
7	Создали составные и частичные индексы под условия запросов.	Чтобы ускорить фильтрацию, соединение и дать возможность Index Only Scan.
8	Сняли планы после оптимизации.	Чтобы доказать, что индексы

		реально используются PostgreSQL.
9	Провели 5 измерений до/после и посчитали среднее, ускорение и процент улучшения.	Чтобы выполнить требование отчета и снизить влияние случайного шума.
10	Построили графики и подготовили отчет.	Чтобы визуально показать изменение времени и стабильность результатов.

3. Теоретическая база

3.1. Логический, предварительный, физический и реальный план

Логический план описывает, какие реляционные операции нужны для получения результата: выборка, проекция, соединение, группировка, сортировка, проверка существования. Он не привязан к конкретному способу чтения таблиц.

Физический план показывает, как именно PostgreSQL будет выполнять логические операции: Seq Scan, Index Scan, Bitmap Heap Scan, Hash Join, Sort, HashAggregate и другие узлы.

Предварительный план получается командой EXPLAIN без фактического выполнения. Реальный план получается через EXPLAIN ANALYZE: запрос реально исполняется, а PostgreSQL показывает фактическое время, количество строк и loops.

В отчете использовался EXPLAIN (ANALYZE, BUFFERS, VERBOSE), поэтому в одном выводе видны и оценочные стоимости, и фактические метрики выполнения.

3.2. Основные узлы планов, которые встретились

Узел плана	Смысл	Как интерпретировать
Seq Scan	Последовательное чтение таблицы.	Плохо для селективной выборки из большой таблицы, нормально для маленькой таблицы или низкой селективности.
Parallel Seq Scan	Последовательное чтение с параллельными worker-процессами.	Может быть быстрее обычного Seq Scan, но все равно читает большую часть таблицы.
Bitmap Index Scan	Поиск подходящих строк по индексу с построением bitmap.	Полезен, когда нужно выбрать несколько процентов таблицы.
Bitmap Heap Scan	Чтение страниц таблицы по bitmap.	Следует после Bitmap Index Scan, потому что часть данных нужно взять из heap.
Index Only Scan	Чтение данных только из индекса без обращения к строкам таблицы.	Быстро, если все нужные поля есть в индексе и visibility map позволяет избежать heap fetch.
Hash Join	Соединение через построение хеш-таблицы.	Типично для соединений по равенству на достаточно больших наборах.
Hash Semi Join	Полусоединение для EXISTS.	Проверяет наличие связанной строки, не размножая строки результата.
Sort	Сортировка результата.	Остается даже после индексов, если порядок индекса не полностью покрывает ORDER BY.
HashAggregate / GroupAggregate	Группировка и агрегация.	Стоимость зависит от объема

		промежуточного результата.
--	--	----------------------------

3.3. Индексы: почему они ускоряют, но не всегда одинаково

Индекс ускоряет не сам запрос целиком, а доступ к строкам. Если после поиска строк еще нужно выполнить JOIN, GROUP BY, Sort или построить hash-таблицу, эти операции остаются в плане и продолжают занимать время.

Составной индекс эффективен, когда порядок полей соответствует условиям запроса. Обычно первыми ставят поля с равенством, затем диапазонные поля и поля для соединения/сортировки.

Частичный индекс содержит только часть таблицы, например только секретные документы или только несекретные документы. Это уменьшает индекс и делает его более точным для конкретного запроса.

INCLUDE-поля позволяют добавить в индекс атрибуты, которые нужны в SELECT, но не должны участвовать в ключе. Это повышает шанс на Index Only Scan.

Селективность показывает, какую долю таблицы выбирает условие. Чем меньше доля строк результата, тем выгоднее индекс. Если условие возвращает значительную часть таблицы, Seq Scan может оказаться не хуже или даже лучше индексного доступа.

4. Запросы Q1-Q3: назначение, узкие места, оптимизация

4.1. Q1: выборка секретных действующих документов

Q1 выбирает секретные документы из нескольких подразделений с действующим сроком. Это селективная выборка из одной таблицы documents.

Запрос Q1

```
SELECT document_number, document_type, responsible_employee,
       internal_department, effective_from, effective_until, secrecy_level
FROM documents
WHERE internal_department IN ('Подразделение_3', 'Подразделение_7', 'Подразделение_13')
  AND secrecy_level = 'секретно'
  AND effective_until IS NOT NULL
  AND effective_until >= DATE '2024-01-01'
ORDER BY effective_until, document_number;
```

Исходное узкое место: Seq Scan по всей таблице documents. PostgreSQL прочитал 100000 строк, вернул 2060 строк и отбросил фильтром 97940 строк. Это означает высокую селективность: результат составляет примерно 2.06% таблицы.

Индекс Q1

```
CREATE INDEX idx_documents_q1_secret_department_until
ON documents (secrecy_level, internal_department, effective_until, document_number)
INCLUDE (document_type, responsible_employee)
WHERE secrecy_level IS NOT NULL
```

```
AND effective_until IS NOT NULL;
```

После оптимизации план использует Bitmap Index Scan и Bitmap Heap Scan. Ускорение получилось самым большим, потому что индекс заменил полный просмотр таблицы на поиск по условиям фильтрации.

4.2. Q2: агрегация операций сотрудника за период

Q2 соединяет document_operations и documents, отбирает операции конкретного сотрудника за период, фильтрует секретные документы, группирует результат и сортирует его.

Запрос Q2

```
SELECT d.responsible_employee, d.internal_department, count(*) AS operations_count
FROM document_operations AS o
JOIN documents AS d ON d.document_number = o.document_number
WHERE o.employee_name = 'Сотрудник_В'
AND o.change_date >= TIMESTAMP '2024-06-01 00:00:00'
AND o.change_date < TIMESTAMP '2024-09-01 00:00:00'
AND d.secrecy_level IS NOT NULL
GROUP BY d.responsible_employee, d.internal_department
ORDER BY operations_count DESC, d.responsible_employee, d.internal_department;
```

Исходные узкие места: Parallel Seq Scan по document_operations и documents, затем Parallel Hash Join. Индекс не может убрать всю стоимость, потому что после отбора все равно остаются JOIN, GROUP BY и ORDER BY.

Индексы Q2

```
CREATE INDEX idx_operations_q2_employee_date_document
ON document_operations (employee_name, change_date, document_number);

CREATE INDEX idx_documents_q2_secret_document_employee_department
ON documents (document_number, responsible_employee, internal_department)
WHERE secrecy_level IS NOT NULL;
```

После оптимизации план использует Index Only Scan по обеим таблицам и Hash Join. Ускорение умеренное, потому что индекс ускоряет чтение, но соединение и агрегация остаются.

4.3. Q3: поиск несекретных документов с изменением срока действия

Q3 ищет несекретные документы, для которых существует операция «Изменение срока действия» за период. Условие EXISTS оптимизатор реализует как Hash Semi Join.

Запрос Q3

```
SELECT d.internal_department, count(*) AS changed_documents
FROM documents AS d
WHERE d.secrecy_level IS NULL
AND EXISTS
(
  SELECT 1
```

```
FROM document_operations AS o
WHERE o.document_number = d.document_number
AND o.operation_description = 'Изменение срока действия'
AND o.change_date >= TIMESTAMP '2024-01-01 00:00:00'
AND o.change_date < TIMESTAMP '2025-01-01 00:00:00'
)
GROUP BY d.internal_department
ORDER BY changed_documents DESC, d.internal_department;
```

Исходные узкие места: Parallel Seq Scan по documents и document_operations, затем Parallel Hash Semi Join. Запрос тяжелее Q1, потому что есть EXISTS, группировка и сортировка.

Индексы Q3

```
CREATE INDEX idx_operations_q3_operation_date_document
ON document_operations (operation_description, change_date, document_number);

CREATE INDEX idx_documents_q3_public_document_department
ON documents (document_number, internal_department)
WHERE secrecy_level IS NULL;
```

После оптимизации план использует Index Only Scan по операциям и частичный индекс по несекретным документам. Ускорение около двух раз реалистично: индекс уменьшает стоимость чтения, но Hash Semi Join и GROUP BY остаются.

5. Расчеты ускорения и проверка реалистичности

Ускорения рассчитаны не по одному фрагменту EXPLAIN ANALYZE, а по отдельной серии из пяти запусков после прогревочного выполнения. Фрагменты EXPLAIN в отчете показывают структуру плана, а таблицы измерений используются для расчета средних времен.

Формулы

```
Коэффициент ускорения = T_before / T_after
Снижение времени, % = (T_before - T_after) / T_before * 100%
```

Запрос	До, ms	После, ms	Ускорение	Снижение, %	Причина результата
Q1	11.494	2.335	4.923	79.69	Селективная выборка из одной таблицы; 2060 строк из 100000, Seq Scan заменен на Bitmap Index Scan + Bitmap Heap Scan.
Q2	21.446	10.519	2.039	50.95	Индексы ускорили доступ, но остались Hash

					Join, группировка и сортировка.
Q3	40.708	19.614	2.075	51.82	Индексы ускорили чтение, но остались Hash Semi Join, GROUP BY и ORDER BY.

5.1. Подробный расчет Q1

До оптимизации:

$(12.791 + 10.760 + 11.604 + 11.672 + 10.642) / 5 = 11.4938 \text{ ms} \approx 11.494 \text{ ms}$

После оптимизации:

$(2.028 + 3.337 + 2.456 + 1.926 + 1.927) / 5 = 2.3348 \text{ ms} \approx 2.335 \text{ ms}$

Ускорение:

$11.494 / 2.335 = 4.9225 \approx 4.923 \text{ раза}$

Снижение времени:

$(11.494 - 2.335) / 11.494 * 100\% = 79.69\%$

Селективность Q1:

$2060 / 100000 * 100\% = 2.06\%$

Отброшено фильтром:

$97940 / 100000 * 100\% = 97.94\%$

Q1 ускорился сильнее всего, потому что запросу нужно около 2% таблицы, а без индекса PostgreSQL читал 100% таблицы. Ускорение не равно 48.5 раза, потому что остаются bitmap, чтение heap-страниц, сортировка и накладные расходы выполнения.

5.2. Подробный расчет Q2

До оптимизации:

$(24.745 + 19.851 + 19.170 + 23.652 + 19.813) / 5 = 21.4462 \text{ ms} \approx 21.446 \text{ ms}$

После оптимизации:

$(12.157 + 9.471 + 9.680 + 10.746 + 10.541) / 5 = 10.519 \text{ ms}$

Ускорение:

$21.446 / 10.519 = 2.0388 \approx 2.039 \text{ раза}$

Снижение времени:

$(21.446 - 10.519) / 21.446 * 100\% = 50.95\%$

Q2 ускорился примерно в 2 раза, потому что индексы улучшили доступ к строкам, но запрос продолжает выполнять соединение, агрегацию и сортировку. Поэтому результат реалистичен.

5.3. Подробный расчет Q3

До оптимизации:

$(35.672 + 43.794 + 40.446 + 38.531 + 45.099) / 5 = 40.7084 \text{ ms} \approx 40.708 \text{ ms}$

После оптимизации:

$(20.921 + 20.259 + 18.557 + 19.138 + 19.197) / 5 = 19.6144 \text{ ms} \approx 19.614 \text{ ms}$

Ускорение:

$40.708 / 19.614 = 2.0754 \approx 2.075$ раза

Снижение времени:

$(40.708 - 19.614) / 40.708 * 100\% = 51.82\%$

Q3 ускорился примерно так же, как Q2, потому что это не простая выборка. EXISTS преобразуется в Hash Semi Join, а группировка и сортировка остаются в плане.

5.4. Насколько результаты похожи на реальность

По относительным ускорениям результаты выглядят реалистично. Для селективного запроса из одной таблицы Q1 ускорение почти в 5 раз нормально. Для запросов с JOIN, EXISTS, GROUP BY и ORDER BY ускорение около 2 раз также типично: индексы ускоряют доступ к данным, но не устраняют стоимость соединения и агрегации.

Слабое место - абсолютное время. Методичка рекомендует ориентироваться на запросы дольше 2-3 секунд, а у нас времена измеряются миллисекундами. Это связано с тем, что 100000/300000 строк для PostgreSQL на локальном ПК и с прогретым кэшем - небольшой объем. Для строгого выполнения этого пункта нужно увеличивать объем данных или выполнять серию запросов в одном измерении.

6. Методические риски и как их честно защищать

Риск	Что сказать на защите
Запросы не являются буквально запросами из ЛР3.	Да, запросы Q1-Q3 разработаны отдельно к базе ЛР1 по варианту 8, потому что ЛР3 была недоступна/не согласована. Они покрывают требуемые операции для ЛР5: фильтрацию, JOIN, EXISTS, GROUP BY, ORDER BY.
Запросы выполняются за миллисекунды, а не за 2-3 секунды.	Минимальный объем данных превышен: 100000 и 300000 строк. На текущем оборудовании PostgreSQL выполняет эти запросы быстро. Поэтому сравнение сделано по 5 измерениям и по изменению физического плана. Если нужно строго попасть в секунды, объем данных надо увеличивать.
Почему значения в листингах EXPLAIN отличаются от средних в таблицах.	Листинги EXPLAIN показывают отдельные запуски для демонстрации структуры плана. Итоговые ускорения посчитаны по отдельной серии из пяти запусков с прогревом. Разница нормальна из-за кэша, состояния буферов, Docker/WSL и параллелизма.
Почему Q2/Q3 ускорились меньше Q1.	В Q2/Q3 индексы ускорили чтение, но не убрали Hash Join/Hash Semi Join, GROUP BY и ORDER BY. Поэтому ускорение меньше, чем у селективного Q1.

7. На основе чего сделаны выводы

Выводы сделаны не только по времени, но и по изменению планов выполнения. Это важно, потому что время в миллисекундах зависит от кэша, но смена Seq Scan на Bitmap Index Scan или Index Only Scan показывает реальную работу оптимизатора.

Вывод	Основание
Q1 был узким из-за полного сканирования documents.	В исходном плане: Seq Scan on public.documents, Rows Removed by Filter: 97940, returned rows: 2060.
Индекс Q1 сработал.	В плане после оптимизации: Bitmap Index Scan on idx_documents_q1_secret_department_until + Bitmap Heap Scan.
Q2 ускорился за счет индексного доступа.	После оптимизации появились Index Only Scan по idx_operations_q2_employee_date_document и idx_documents_q2_secret_document_employee_department.
Q3 оптимизатор реализовал EXISTS как полусоединение.	В плане: Hash Semi Join / Parallel Hash Semi Join.
Ускорения посчитаны корректно.	Файл report/lab5_measurements_summary.csv и таблица 6 отчета: средние значения по 5 запускам.
Графики построены на основе CSV.	Файлы charts/lab5_avg_execution_time.png, lab5_speedup_ratio.png, lab5_improvement_percent.png, lab5_q1_runs.png, lab5_q2_runs.png, lab5_q3_runs.png.

8. Команды для проверки и демонстрации в консоли

Команды ниже рассчитаны на запуск из WSL в папке Lab5. Их можно использовать для быстрой проверки стенда и демонстрации преподавателю.

8.1. Быстрая проверка, что стенд живой

```
cd /mnt/c/Users/Georgul/Documents/8_sem/SUBD/Lab5

docker compose up -d
docker compose ps

docker exec subd_lab5_postgres pg_isready -U lab5_admin -d subd_lab5

./scripts/run_sql.sh sql/03_check_counts.sql logs/demo_03_check_counts.log

cat report/lab5_measurements_summary.csv
ls -lh report/*.csv charts/*.png logs/*.log
```

8.2. Показать количество строк, размеры и индексы

```
PGPASSWORD=lab5_admin_pass psql \
-h 127.0.0.1 \
-p 25432 \
-U lab5_admin \
-d subd_lab5 \
-P pager=off \
-c "SELECT 'documents' AS table_name, count(*) FROM documents
UNION ALL
```

```
SELECT 'document_operations', count(*) FROM document_operations;"
```

```
PGPASSWORD=lab5_admin_pass psql \  
-h 127.0.0.1 \  
-p 25432 \  
-U lab5_admin \  
-d subd_lab5 \  
-P pager=off \  
-c "SELECT relname,  
      pg_size_pretty(pg_relation_size(oid)) AS table_size,  
      pg_size_pretty(pg_total_relation_size(oid)) AS total_size  
FROM pg_class  
WHERE relname IN ('documents', 'document_operations')  
ORDER BY relname;"
```

```
PGPASSWORD=lab5_admin_pass psql \  
-h 127.0.0.1 \  
-p 25432 \  
-U lab5_admin \  
-d subd_lab5 \  
-P pager=off \  
-c "SELECT tablename, indexname  
FROM pg_indexes  
WHERE tablename IN ('documents', 'document_operations')  
ORDER BY tablename, indexname;"
```

8.3. Полный воспроизводимый прогон с нуля

Эти команды пересоздают таблицы и данные. Запускать, если нужно показать полностью воспроизводимый сценарий.

```
cd /mnt/c/Users/Georgul/Documents/8_sem/SUBD/Lab5  
  
docker compose up -d  
  
./scripts/run_sql.sh sql/00_reset.sql logs/demo_00_reset.log  
./scripts/run_sql.sh sql/01_schema.sql logs/demo_01_schema.log  
./scripts/run_sql.sh sql/02_fill_test_data.sql logs/demo_02_fill_test_data.log  
./scripts/run_sql.sh sql/03_check_counts.sql logs/demo_03_check_counts.log
```

8.4. Демонстрация планов до и после индексов

Этот сценарий сначала удаляет оптимизирующие индексы, показывает исходные планы, затем создает индексы и показывает оптимизированные планы.

```
cd /mnt/c/Users/Georgul/Documents/8_sem/SUBD/Lab5  
  
cat > sql/demo_drop_opt_indexes.sql <<'SQL'  
\encoding UTF8  
\pset pager off  
\timing on  
DROP INDEX IF EXISTS idx_documents_q1_secret_department_until;  
DROP INDEX IF EXISTS idx_operations_q2_employee_date_document;  
DROP INDEX IF EXISTS idx_operations_q3_operation_date_document;  
DROP INDEX IF EXISTS idx_documents_q3_public_document_department;
```

```

DROP INDEX IF EXISTS idx_documents_q2_secret_document_employee_department;
ANALYZE documents;
ANALYZE document_operations;
SELECT 'Оптимизирующие индексы удалены' AS result;
SQL

./scripts/run_sql.sh sql/demo_drop_opt_indexes.sql logs/demo_drop_opt_indexes.log
./scripts/run_sql.sh sql/04_baseline_plans.sql logs/demo_04_baseline_plans.log

./scripts/run_sql.sh sql/05_create_indexes.sql logs/demo_05_create_indexes.log
./scripts/run_sql.sh sql/06_optimized_plans.sql logs/demo_06_optimized_plans.log

printf '\n=== BASELINE: ключевые операции ===\n'
grep -E "Seq Scan | Parallel Seq Scan | Bitmap Index Scan | Bitmap Heap Scan | Index Only Scan | Hash Join | Hash Semi Join | Execution Time" logs/demo_04_baseline_plans.log

printf '\n=== OPTIMIZED: ключевые операции ===\n'
grep -E "Seq Scan | Parallel Seq Scan | Bitmap Index Scan | Bitmap Heap Scan | Index Only Scan | Hash Join | Hash Semi Join | Execution Time" logs/demo_06_optimized_plans.log

```

8.5. Быстрая демонстрация Q1 в интерактивном psql

```

PGPASSWORD=lab5_admin_pass psql \
-h 127.0.0.1 \
-p 25432 \
-U lab5_admin \
-d subd_lab5 \
-P pager=off

-- Внутри psql:
SET jit = off;
EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
SELECT
    document_number,
    document_type,
    responsible_employee,
    internal_department,
    effective_from,
    effective_until,
    secrecy_level
FROM documents
WHERE internal_department IN ('Подразделение_3', 'Подразделение_7', 'Подразделение_13')
AND secrecy_level = 'секретно'
AND effective_until IS NOT NULL
AND effective_until >= DATE '2024-01-01'
ORDER BY effective_until, document_number;

-- В плане после индексов должны быть Bitmap Index Scan / Bitmap Heap Scan.
-- Выход из psql:
\q

```

8.6. Пересчет измерений и экспорт CSV

Если файл `sql/07_measure_5_runs.sql` в текущем репозитории содержит старую ошибку `\copy` в конце, он все равно успевает создать таблицу измерений до ошибки. Для чистой демонстрации можно использовать уже сохраненный `report/lab5_measurements_summary.csv`. Если нужно пересчитать, выполните 07, затем 08.

```
cd /mnt/c/Users/Georgul/Documents/8_sem/SUBD/Lab5

./scripts/run_sql.sh sql/07_measure_5_runs.sql logs/demo_07_measure_5_runs.log || true
./scripts/run_sql.sh sql/08_export_measurements.sql logs/demo_08_export_measurements.log

cat report/lab5_measurements_summary.csv
python3 scripts/plot_measurements.py
ls -lh charts/*.png report/*.csv
```

8.7. Что показывать преподавателю по порядку

10. Показать, что контейнер живой: `docker compose ps` и `pg_isready`.
11. Показать объем данных: 100000 documents и 300000 document_operations.
12. Показать исходный план Q1 без индексов: Seq Scan и Rows Removed by Filter.
13. Показать созданный индекс Q1 и план после оптимизации: Bitmap Index Scan + Bitmap Heap Scan.
14. Показать Q2/Q3: до - Parallel Seq Scan, после - Index Only Scan.
15. Открыть `report/lab5_measurements_summary.csv` и объяснить формулу ускорения.
16. Показать графики `charts/lab5_avg_execution_time.png` и `charts/lab5_speedup_ratio.png`.
17. Честно сказать, что Q1-Q3 разработаны под предметную область ЛР1, потому что ЛР3 была недоступна, и что абсолютные времена малы из-за локального кэша и быстрого оборудования.

9. Короткие ответы на типовые вопросы преподавателя

Вопрос	Короткий ответ
Почему Q1 ускорился сильнее всего?	Потому что это селективная выборка из одной таблицы: вернулось 2060 строк из 100000, и Seq Scan заменился на Bitmap Index Scan.
Почему Q2/Q3 ускорились только примерно в 2 раза?	Потому что кроме чтения строк там остаются JOIN/EXISTS, GROUP BY и ORDER BY. Индекс ускоряет доступ, но не отменяет обработку промежуточного результата.
Почему Index Only Scan не обращается к таблице?	Потому что нужные поля есть в индексе, а Heap Fetches = 0 показывает, что PostgreSQL не ходил в heap за строками.
Почему используется Hash Semi Join в Q3?	Потому что EXISTS проверяет наличие связанной строки, а не требует выводить все совпадения. Это логически полусоединение.
Почему не Nested Loop?	Для больших наборов и соединения по равенству PostgreSQL часто выбирает Hash Join, потому что построить hash-таблицу дешевле, чем многократно искать строки во вложенном цикле.
Почему времена из EXPLAIN и из таблицы разные?	EXPLAIN-фрагменты - одиночные запуски для

	структуры плана; итоговые таблицы - среднее по пяти запускам с прогревом.
Что является главным методическим риском?	Запросы не буквально из ЛРЗ, а абсолютные времена меньше 2-3 секунд. Но планы и относительные ускорения корректны и воспроизводимы.

10. Используемые материалы

18. Практикум по построению защищенных баз данных: учебное пособие / М. А. Полтавцева и др. - СПб.: ПОЛИТЕХ-ПРЕСС, 2023.
19. PostgreSQL Documentation: EXPLAIN.
20. PostgreSQL Documentation: Indexes.
21. PostgreSQL Documentation: Partial Indexes.
22. PostgreSQL Documentation: Index-Only Scans and Covering Indexes.
23. Файлы проекта ЛР5: sql/*.sql, logs/*.log, report/*.csv, charts/*.png, отчет ЛР5.